

National University of Singapore
School of Computing
CS1101S: Programming Methodology
Semester I, 2026/2027

S6

Problem Solving and Linked List Processing

LINKED LISTS Functions of Python §2

The LINKED LISTS functions are a set of predeclared functions provided in Python §2 for linked list processing. Please refer to the [online reference](#) for details of these LINKED LISTS functions.

Problems:

1. Write the function `map` **using reduce**. Name your function `my_map`.

```
def my_map(f, xs):  
    # this should be a one-liner  
    ...
```

Example calls:

```
my_map(lambda x: x + 1, llist(1, 2, 3))  
# Result: llist(2, 3, 4)
```

2. Write a function called `remove_duplicates` that takes in a `llist` as its only argument and returns a `llist` with duplicate elements removed. The order of the elements in the returned `llist` does not matter. **Use filter in your function.**

```
def remove_duplicates(lst):  
    ...
```

Example calls:

```
remove_duplicates(llist(1, 2, 3, 4, 4, 3, 2, 1, 2))  
# Result: llist(1, 2, 3, 4)  
  
remove_duplicates(llist("a", "x", "b", "c", "c", "b", "d"))  
# Result: llist("a", "x", "b", "c", "d")
```

3. Our friend Louis Reasoner has a pocket full of change. He wants to buy a snack that will cost him x cents, and he wants to know all the ways in which he can use his change to make up that amount. Please help him in writing a function which takes as parameters the amount x and a `llist` of all the coins Louis has in his pocket, and returns a `llist` of `llists`, such that each sub-`llist` of the result contains a valid combination to make up x . A

combination may appear more than once, since it may be using different coins of the same denomination.

Help Louis by filling in the ellipses ... in his incomplete solution:

```
def makeup_amount(x, coins):
    if x == 0:
        return llist(None)
    elif x < 0 or is_none(coins):
        return None
    else:
        # Combinations that do not use the head coin.
        combi_A = ...

        # Combinations that do not use the head coin
        # for the remaining amount.
        combi_B = ...

        # Combinations that use the head coin.
        combi_C = ...

    return append(combi_A, combi_C)
```

Example call:

```
makeup_amount(22, llist(1, 10, 5, 20, 1, 5, 1, 50))
# Result: llist(llist(20, 1, 1), llist(10, 5, 1, 5, 1), llist(1, 20, 1),
#           llist(1, 20, 1), llist(1, 10, 5, 5, 1),
#           llist(1, 10, 5, 1, 5))
```

Note: The sub-llist `llist(1, 20, 1)` appears twice. Each appearance of the number 1 refers to a different coin.